Angular Integration (existing app, 4 steps)

1. `npm i @pixous/ai-suite @ngx-translate/core @ngx-translate/http-loader` (or copy `angular/pixous-ai-suite/src/lib` + `assets/i18n` into `shared/`).
2. Provide backend URL + TranslateModule once (`app.config.ts`): ``typescript import { AI_SUITE_CONFIG } from '@pixous/ai-suite'; // + TranslateModule.forRoot({ defaultLanguage: 'en', // loader: { provide: TranslateLoader, useFactory: httpLoaderFactory, deps: [HttpClient] } }) { provide: AI_SUITE_CONFIG, useValue: { apiBaseUrl: 'https://api.myapp.com' } } ``
3. Drop widgets where needed (all inputs plain data — no secrets): ``html <pix-chat-widget title="Pixous Assistant" responseLanguage="ta" [quickReplies]="['Our Services','Products','Contact','Careers']" /> <pix-language-switcher /> ` Component API: title, placeholder, responseLanguage (backend allow-listed), quickReplies (set [] to hide). Theme via .pix-*` CSS.
4. Copy `assets/i18n/*.json` to `src/assets/i18n/`, add your keys, init once: `i18n.init('en', ['en','ta','hi','es','fr'])`.

Services cheat sheet

Member	Does
`AiChatService.send(msg, {responseLanguage?, provider?, sessionId?})`	POST `/api/ai/chat`, updates `messages$/typing$`
`AiChatService.stream(msg, opts?, signal?)`	SSE `/api/ai/chat/stream` as `AsyncGenerator<string>`
`AiChatService.getSelection()/setSelection()`	Live provider switching (admin UI)
`AiTranslationService.use()/instant()`	Runtime UI strings via ngx-translate
`AiTranslationService.translate()/translateBatch()/detect()`	Backend translation (envelope-unwrapped)
`AiTranslationService.supportedLanguages()`	Backend provider's languages
`aiSuiteAuthInterceptor(getToken)`	Attaches YOUR app token to `/api/ai/*` (never an AI key)